



武漢大學
WUHAN UNIVERSITY

RuOS

决赛设计文档

参 赛 队 名 : _____ RUOK

队 伍 成 员 : _____ 干瑞麟 周锦耀 黄文婷

指 导 老 师 : _____ 蔡朝晖

二〇二六 年 一 月

摘要

RuOS 是一个使用 C 语言实现，支持 RISCv64 硬件平台的多核宏内核操作系统。RuOS 基于 xv6 操作系统，并在进程管理、内存管理、文件系统、信号机制、网络模块、系统调用等方面进行了大量改进和优化，提升了系统的性能和稳定性。本文档详细介绍了 RuOS 的设计理念、架构、实现细节以及测试结果，展示了其在多核处理器环境下的高效、稳定的运行能力。

RuOS 各个模块的具体改进如下表所示：

模块	改进内容
进程管理	引入多级反馈队列调度算法，实现简单的负载均衡
内存管理	Buddy + Slab 分配器结合，提升内存分配效率
内存管理	支持写时复制、零页分配、懒分配，减少内存分配的时间开销
文件系统	通过类 VFS 设计提供对 FAT32、EXT4 文件系统的支持
信号机制	实现类 Linux 信号子系统，pending/屏蔽字与用户态 handler，提供 <code>rt_sigaction</code> / <code>rt_sigprocmask</code> / <code>rt_sigtimedwait</code> / <code>rt_sigreturn</code> / <code>kill_signal</code> 等系统调用
网络模块	集成 ONPS TCP/IP 协议栈，支持 TCP/UDP 协议，支持协议栈内回环以及 tap 模式下与宿主机通信
程序装载	完善 exec/ELF 装载与动态链接兼容：修正用户栈 argv/envp/auxv 布局与对齐，支持 <code>PT_INTERP</code> 解释器装载与路径回退，补齐 <code>AT_*</code> auxv 并实现 <code>mprotect</code> （RELRO）
系统调用	按 LINUX 语义实现或简化实现几十种系统调用，按 LINUX 语义返回错误码，为 busybox 等用户态程序提供内核支持
设备驱动	完善 PLIC 中断分发与设备驱动支持，如串口与 virtio 磁盘/网卡设备

RuOS 通过了初赛的所有系统调用测试，初赛得分为 102/102:

开始评测时间: 2026-01-14 13:47:18.580323+08:00

结束评测时间: 2026-01-14 13:47:27.726123+08:00

得分: 102

目录

1	RuOS 架构图	5
2	进程间通信	6
2.1	信号机制	6
2.1.1	信号来源与语义	6
2.1.2	关键数据结构	6
2.1.3	发送与递送流程	7
2.1.4	用户态 handler 构造与返回	7
3	网络模块	9
3.1	QEMU 网络模式	9
3.1.1	User Networking (SLIRP)	9
3.1.2	Tap Networking	10
3.2	设备层	11
3.3	网络层：以太网、ARP 与 IPv4	12
3.4	传输层：UDP/TCP	13
3.5	数据路径	14
4	程序装载	15
4.1	ELF 文件格式	15
4.2	用户栈的初始化与参数传递	16
5	系统调用	18
5.1	调用路径概览	19
5.2	TRAMPOLINE 与 trapframe 机制	20
5.3	参数传递与用户指针解码	21
5.4	系统调用分发与返回	21
5.5	系统调用集合	21
5.6	错误码与语义对齐	22
5.7	调试与扩展点	23
6	设备驱动	24
6.1	MMIO：设备寄存器与内存序	24
6.2	设备与中断拓扑	24
6.3	UART：控制台与早期调试	25
6.4	PLIC：外部中断控制器	26
6.5	VirtIO：半虚拟化设备与 mmio legacy 接口	26

6.6	virtqueue 与 ring: 从入队到回收	27
6.7	virtio-blk: 块设备 I/O	28
6.8	virtio-net: 收发队列与协议栈对接	28
6.9	工程化细节与可扩展点	28

1 RuOS 架构图

初赛架构图如 图 2 所示：

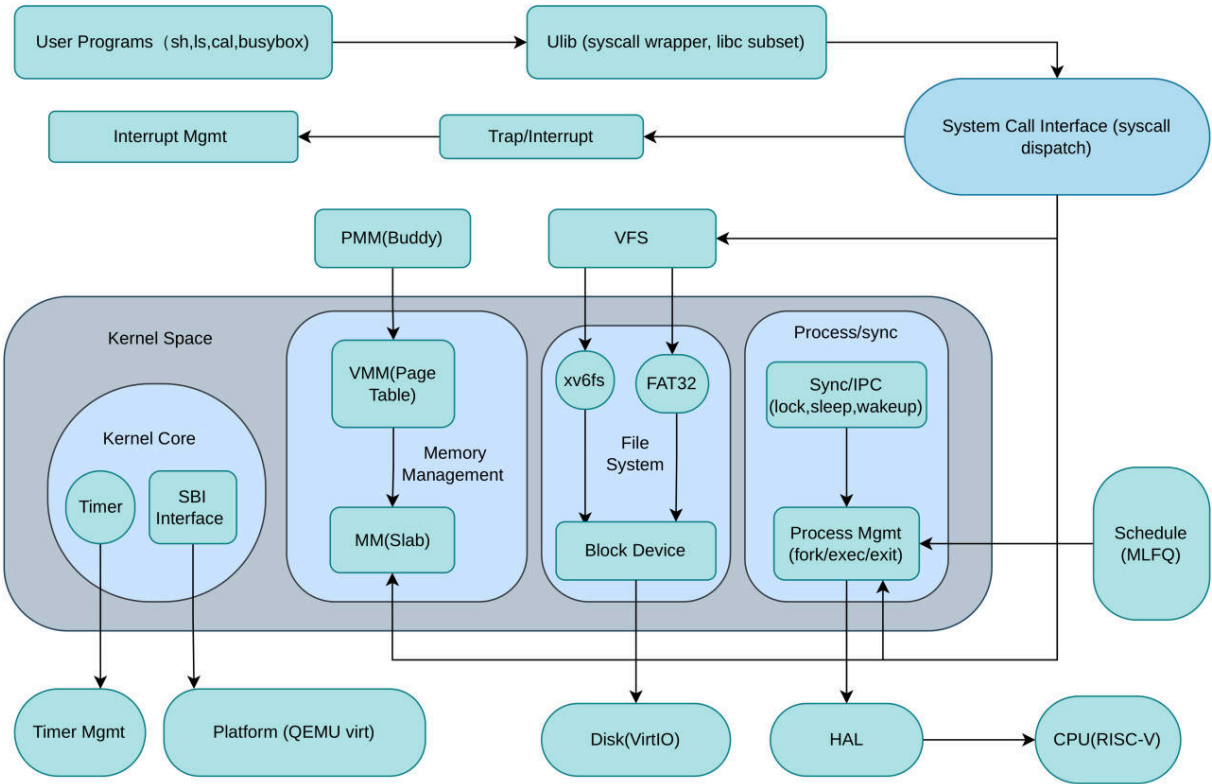


图 2: RuOS 初赛架构图

决赛架构图（草稿）如 图 3 所示：

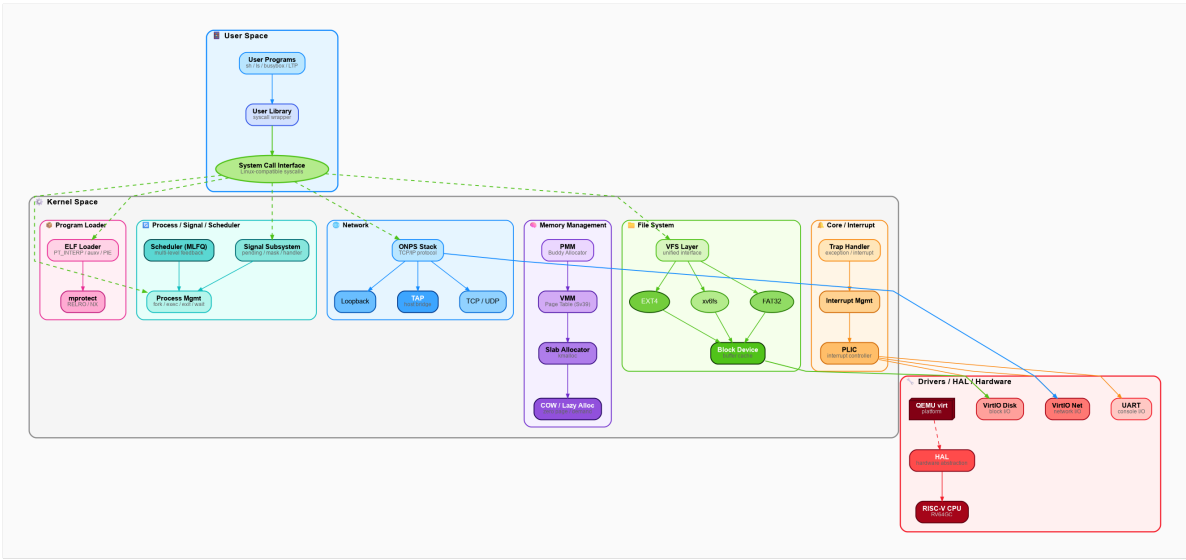


图 3: RuOS 决赛架构图（草稿）

2 进程间通信

2.1 信号机制

RuOS 实现了类 Linux 的信号子系统，支持 pending 信号、屏蔽字以及用户态信号处理函数。内核提供了 `rt_sigaction`、`rt_sigprocmask`、`rt_sigtimedwait`、`rt_sigreturn` 和 `kill_signal` 等系统调用，允许用户进程注册信号处理函数、修改信号屏蔽字、等待信号以及发送信号。信号处理过程遵循 Linux 语义，确保与现有用户态程序的兼容性。

2.1.1 信号来源与语义

信号可以看作是软件层面对中断机制的抽象，主要来源包括：

- 程序错误：如除零、非法内存访问等；
- 外部事件：如终端 Ctrl-C 产生 `SIGINT`、定时器到期产生 `SIGALRM`；
- 显式请求：进程通过 `kill` 发送信号给指定进程或进程组。

与 Linux 保持一致，信号号范围为 `1..64`，并保留非实时信号 (`1..31`) 与实时信号 (`34..64`) 的划分语义。在当前实现中，待处理信号使用 **位图** 表示，使用 `sigpending` 字段进行记录。因此同一信号可能被合并（非实时语义），后续 **可扩展为队列** 以完善实时信号特性。

2.1.2 关键数据结构

1. `struct proc`

在 `proc` 结构体中，与信号相关的字段如下：

```
1 // 信号处理相关
2 uint64 sigpending;           // pending signals bitmap
3 uint64 sigmask;             // blocked signals bitmap
4 struct sigaction sigactions[NSIG]; // per-signal handler settings
```

其中：

- `sigpending`：uint64 位图，记录待处理信号；
- `sigmask`：屏蔽字，表示被阻塞的信号；
- `sigactions[NSIG]`：每个信号的处理动作，包含 `sa_handler`、`sa_mask`、`sa_flags`、`sa_restorer`；

2. `struct sigaction` 与 `struct sigset`

在 `sigaction` 结构体中，定义了每个信号的处理动作：

```
1 struct sigset {
2     uint64 bits;
3 };
4
5 struct sigaction {
6     uint64 sa_handler;
7     uint64 sa_flags;
8     uint64 sa_restorer;
```

```
9 struct sigset sa_mask;
10 };
```

- `sa_handler`：信号处理函数地址，或 `SIG_DFL` / `SIG_IGN`；
- `sa_mask`：处理该信号时额外屏蔽的信号集合；
- `sa_flags`：控制信号处理行为的标志，如 `SA_RESETHAND`、`SA_NODEFER` 等；
- `sa_restorer`：用户态返回内核的地址，通常指向 `rt_sigreturn`。
- `sigframe`：用户栈上的信号帧，保存旧的屏蔽字与 `trapframe`，用于 `rt_sigreturn` 恢复上下文。

用户态可以通过 `rt_sigaction` 系统调用对进程的信号处理行为进行注册。

3. 信号常量

```
1 #define SIG_DFL ((uint64)0) // 默认处理
2 #define SIG_IGN ((uint64)1) // 忽略信号
3 ...
4 #define SIGKILL 9 // 强制终止进程
5 #define SIGSTOP 19 // 停止进程执行
6 ...
```

`SIGKILL` 与 `SIGSTOP` 在任何情况下都不可屏蔽，内核在更新屏蔽字时强制清除这两位。

2.1.3 发送与递送流程

发送信号时，内核通过 `signal_send / signal_send_pid` 校验信号号并设置 `sigpending` 位图，同时将 `SLEEPING` 进程唤醒为 `RUNNABLE`。递送发生在用户态陷入返回之前：`usertrap` 中调用 `signal_handle` 检查并派发信号。

处理逻辑如下：

- 从 `sigpending` 中挑选一个未被 `sigmask` 屏蔽的信号（优先级按信号号递增）；
- `SIG_IGN` 直接忽略；
- `SIG_DFL` 执行默认动作：`SIGCHLD` / `SIGURG` / `SIGWINCH` 默认忽略，其余触发进程终止，并对核心转储类信号设置退出状态 `0x80`；
- 对于用户自定义处理函数，进入用户态 `handler`。

2.1.4 用户态 handler 构造与返回

由于信号处理程序是由用户提供的，所以信号处理程序的代码是在用户态的。而从系统调用返回到用户态前还是属于内核态，CPU 是禁止内核态执行用户态代码的。因此我们需要在用户栈上构造一个信号帧 `sigframe`，并修改 `trapframe` 使得返回到用户态时跳转到用户态的信号处理函数。

`signal_setup_frame` 在用户栈上构造 `sigframe`：

- 16 字节对齐栈指针，写入 `magic`、`old_mask` 与完整 `trapframe`；
- 将 `epc` 指向用户态 `handler`，`a0` 传入信号号，`ra` 设置为 `sa_restorer`；

- 更新 `sigmask`：自动屏蔽当前信号与 `sa_mask`，若设置 `SA_NODEFER` 则不屏蔽当前信号；
- 若设置 `SA_RESETHAND`，处理后自动恢复为默认动作。

用户态处理函数返回后，返回到 `act->sa_restorer`，再通过 `rt_sigreturn` 进入内核，`signal_return` 校验 `sigframe.magic` 并恢复 `trapframe` 与旧屏蔽字，保证控制流回到原用户态执行点。

RuOS 的信号处理流程如图 4 所示：

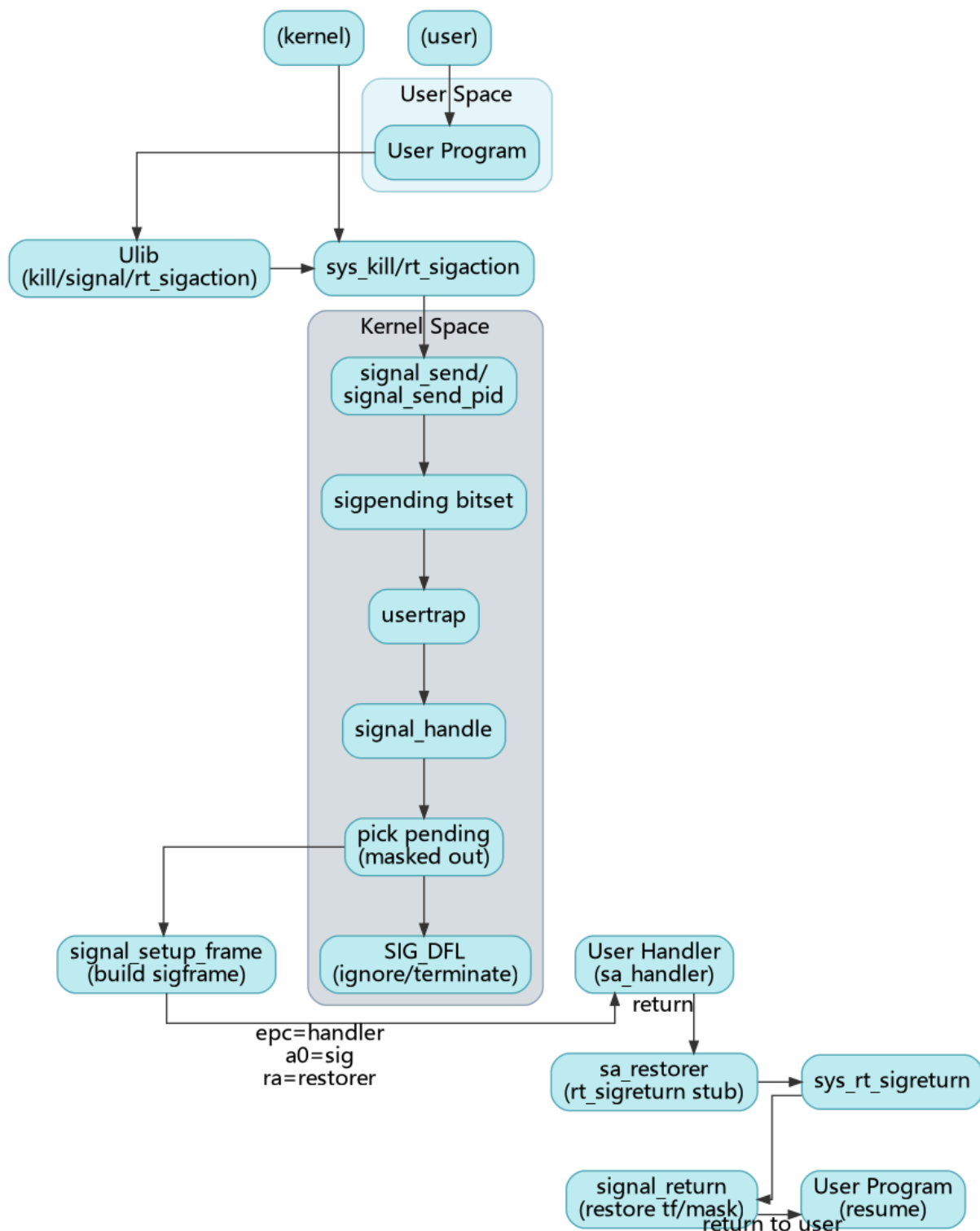


图 4: RuOS 信号处理流程图

3 网络模块

RuOS 集成了 ONPS TCP/IP 协议栈，支持 TCP/UDP 协议。ONPS 协议栈面向资源受限场景，提供完整的 Ethernet/IPv4/TCP/UDP 与 Berkeley socket 层，同时强调 buf list 零拷贝发送 与较低内存占用，保证了高效的网络性能。

在 RuOS 中，我们保留与以太网、IPv4、TCP/UDP 相关的核心功能，关闭 PPP、IPv6 及网络工具等非必需模块，减小内核体积并降低维护成本。RuOS 可通过 virtio-net 设备驱动与虚拟网卡交互，实现数据包的发送与接收。用户态程序可以通过标准的 socket 接口进行网络通信。

3.1 QEMU 网络模式

3.1.1 User Networking (SLIRP)

User Networking (SLIRP) 是 QEMU 的默认网络后端，无需 root 权限即可使用，但存在以下典型限制：

1. 性能较差：由于 SLIRP 在用户态实现完整 TCP/IP 栈，额外拷贝和转换较多，吞吐/延迟不如 tap/bridge。
2. 默认不允许 ICMP：不特殊配置时无法发送或接收 ICMP，因此 ping 通常不可用。
3. 外部无法主动访问虚拟机：缺省没有端口转发 (hostfwd)，宿主机和外网无法直接连接虚拟机。
4. NAT 模式：SLIRP 实现了 NAT 转发，虚拟机访问外部网络是“出站可达”，但“入站默认不可达”。

对应的 qemu 启动参数示例：

- `-device virtio-net-device,netdev=net,bus=virtio-mmio-bus.1`
- `-netdev user,id=net`

对应的默认网络拓扑为：

- 虚拟机 IP: `10.0.2.15`
- 网关: `10.0.2.2`
- DNS: `10.0.2.3`

该拓扑可以用 图 5 表示：

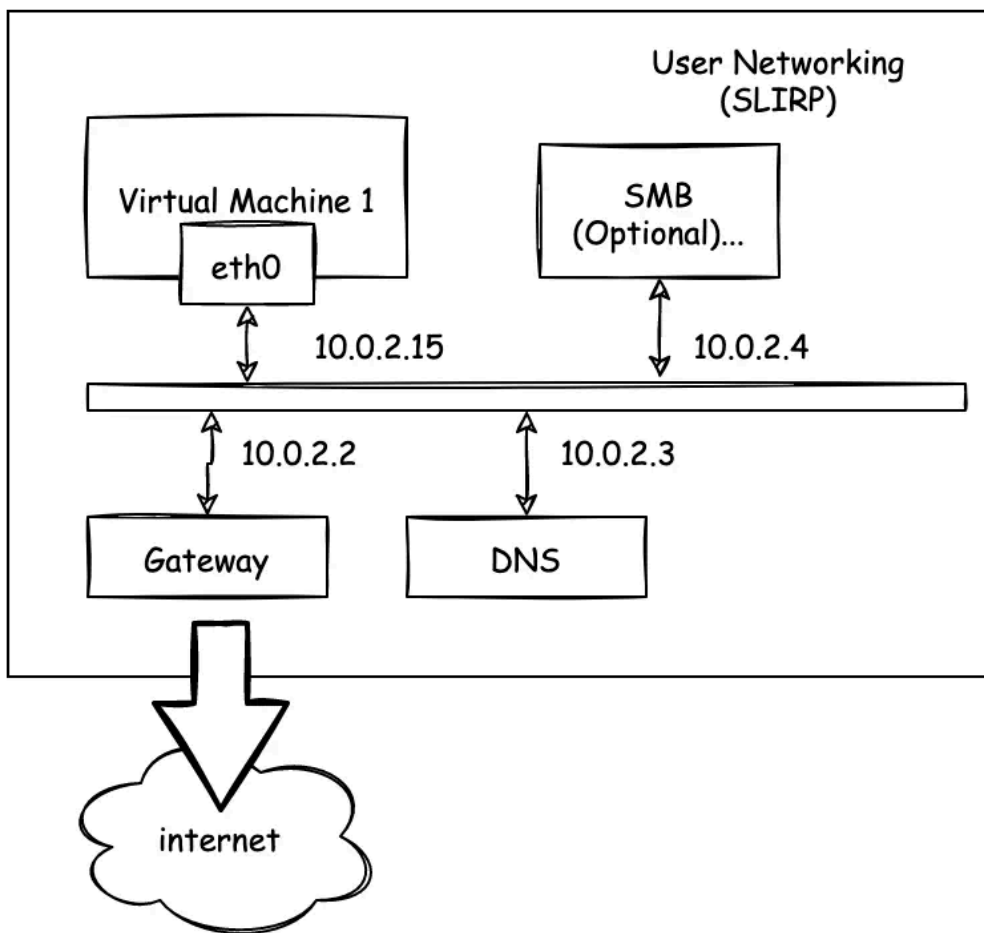


图 5: QEMU User Networking 拓扑图

结合 SLIRP 的限制可以理解：

- 从虚拟机向宿主机发 UDP 包（`10.0.2.2`）可以到达，但如果宿主机没有服务监听，就无法得到回包。
- 想让宿主机主动连虚拟机，需要额外设置 `-netdev user,hostfwd=...`。

3.1.2 Tap Networking

当切换到 tap/bridge 模式时，虚拟机不再经过 SLIRP 的用户态 NAT，而是把二层包直接丢给宿主机桥接设备。这样可以观察更真实的 ARP/TCP 行为，也便于做入站连接或更复杂的网络验证。

对应的一种可能的 qemu 启动参数示例：

- `-device virtio-net-device,netdev=net,bus=virtio-mmio-bus.1`
- `-netdev tap,id=net,ifname=tap0,script=no,downscript=no`

我们需要先在宿主机上配置 tap0 设备并加入桥接，例如：

```
1 sudo ip tuntap add dev tap0 mode tap user $USER
2 sudo ip link set tap0 up
```

Bash

```

3 sudo ip link add br0 type bridge
4 sudo ip link set br0 up
5 sudo ip link set tap0 master br0
6 sudo ip addr add 10.0.2.2/24 dev br0
7 sudo ip link set br0 up

```

该模式下，虚拟机可以直接与宿主机通信，且支持 ICMP 和入站连接。

Tap 模式下的一种可能的网络拓扑如图 6 所示：

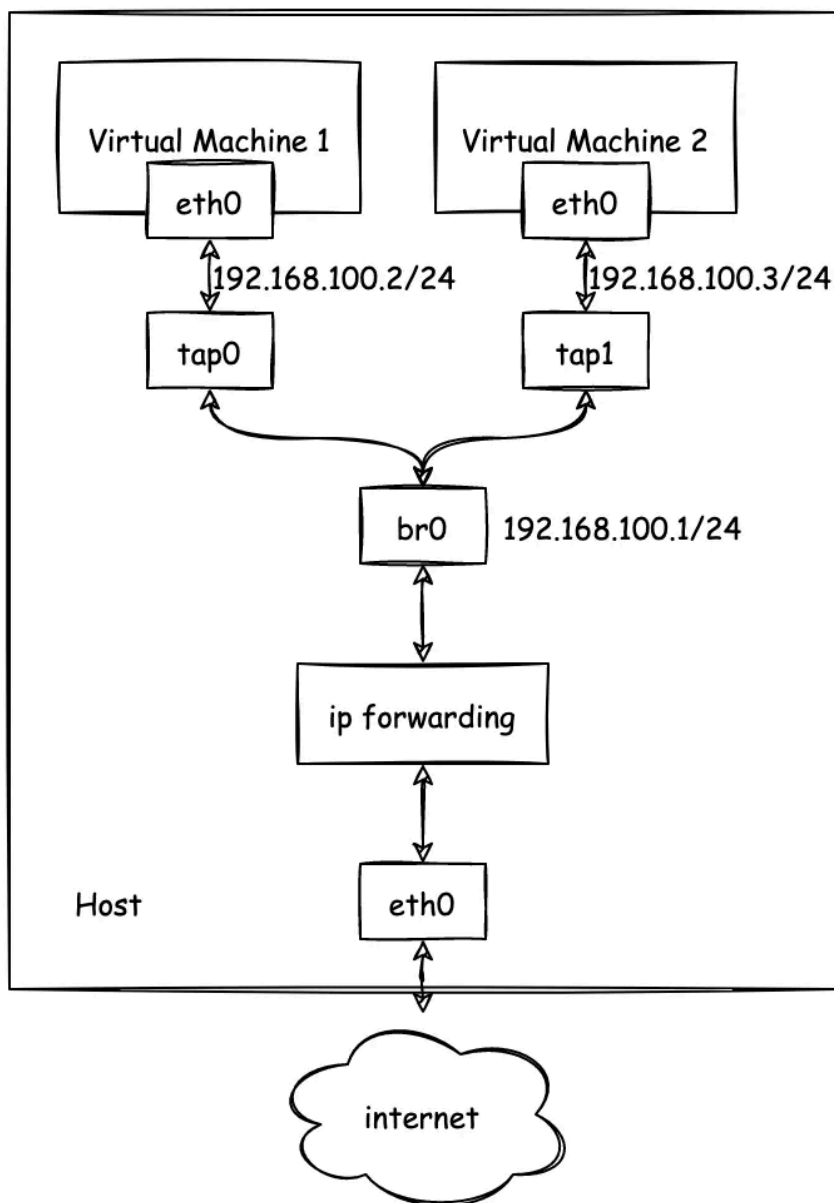


图 6: QEMU Tap Networking 拓扑图

3.2 设备层

设备层负责“把网卡的数据拿进来、把要发的数据送出去”，并保证中断驱动下的持续收发：

1. 初始化：在 virtio-mmio 总线上完成设备发现与特性协商，创建 RX/TX virtqueue；为 RX 队列预分配缓冲区并填充描述符，确保网卡一有数据就能写入；
2. 接收：网卡产生中断后由 PLIC 转发到 `virtio_net_intr`，驱动读取中断状态并确认后进入 `virtio_net_rx`，从 RX used ring 取出已填充的缓冲区，跳过 `virtio_net_hdr` 得到以太网帧，再调用 `net_rx_deliver` 上送协议栈；
3. 发送：`virtio_net_transmit` 把待发数据组织成 TX 描述符链并通知设备；发送完成后在中断里由 `virtio_net_tx_complete` 统一回收；
4. 回收：收包后立即把描述符重新放回 RX avail ring，保证队列不耗尽、收包不断流。

下面是 `net_rx_deliver` 的代码：

```

1  void
2  net_rx_deliver(const uint8 *data, int len)
3  {
4      if (!g_netif || !data || len <= 0)
5          return;
6
7      log_info("net: rx len=%d\n", len);
8      EN_ONPSERR err = ERRNO;
9      PST_SLINKEDLIST_NODE node =
10         (PST_SLINKEDLIST_NODE)buddy_alloc(sizeof(ST_SLINKEDLIST_NODE) +
11         (UINT)len, &err);
12      if (!node) {
13          log_warn("net: rx drop (alloc failed, len=%d, err=%d)\n", len, err);
14          return;
15      }
16      node->uniData.unVal = (UINT)len;
17      memmove(((UCHAR *)node) + sizeof(ST_SLINKEDLIST_NODE), data, len);
18      ethernet_put_packet(g_netif, node);
19  }
```

其中，`data` 指向以太网帧数据，`len` 为帧长度。函数首先检查网络接口和数据有效性，然后分配一个链表节点，将数据复制进去，最后调用 `ethernet_put_packet` 将数据包传递给 ONPS 协议栈进行处理。

通过“预分配缓冲 + 中断驱动 + ring 回收”的机制，设备层为协议栈提供稳定、连续的收发通路。

3.3 网络层：以太网、ARP 与 IPv4

网络层由 ONPS 提供，RuOS 侧需要做的核心是网卡注册与地址配置（接口聚合在 `src/network/net.c`），原因很直接：ONPS 只有在拿到网卡的 MAC/IP、发送回调和接收线程入口后，才能把驱动当作一个可用的 `netif`（Network Interface，网络接口）来管理。

在 ONPS 协议栈中，`ST_IPV4` 结构体如下面代码所示：

```

1  /* 记录IPv4地址的结构体
2  typedef struct _ST_IPV4_ {
3      UINT unAddr;
4      UINT unSubnetMask;
5      UINT unGateway;
6      UINT unPrimaryDNS;
7      UINT unSecondaryDNS;
8      UINT unBroadcast;
9  } ST_IPV4, *PST_IPV4;

```

具体流程如下：

- 在 `net_init` 中先 `open_npstack_load` 启动协议栈，再通过 `virtio_net_init` 获取网卡 MAC；
- 构造 `ST_IPV4`（`10.0.2.15/24` + 网关 `10.0.2.2` + DNS），匹配 QEMU user networking 的默认语义；
- 调用 `ethernet_add` 完成注册：
 - 该函数在 ONPS 内部为网卡分配控制块、ARP 资源与接收队列，并保存发送回调 `net_emac_send`；
 - 同时注册接收线程入口 `net_start_eth_recv_thread`，用于消费由 `net_rx_deliver` 上送的报文。
 - 完成了网卡驱动与 ONPS 协议栈的关键对接。它本质上是创建并初始化了一个 `netif`（网络接口）结构体，并将其添加到 ONPS 的网络接口列表中。

完成注册后，ONPS 就能对 `eth0` 做 ARP 解析、IPv4 解包与封装，并驱动接收线程处理进入协议栈的数据。该层承担“二层帧 → 三层包”的转换，并提供路由与地址解析基础。

3.4 传输层：UDP/TCP

传输层由 ONPS 提供实现，它保证了传输层的基础能力（包含 TCP/UDP 连接管理、数据收发、超时控制、错误码返回等核心能力），RuOS 不重复开发底层能力，仅做接口封装和语义对齐，只选用当前业务需要的功能子集。

1. 功能裁剪与聚焦

ONPS 虽支持 TCP/UDP 全量基础能力，但 RuOS 仅封装实际用到的核心接口：

- UDP：仅封装 `sendto/recvfrom` 接口；
- TCP：仅封装 `connect/listen/accept`（连接管理）和 `send/recv`（数据收发）接口。

2. 系统调用封装实现

网络相关系统调用集中在 `src/syscall/sysnet.c` 文件中实现，核心作用是将用户态传入的参数格式、调用方式，转换为 ONPS API 可识别的格式，对外暴露的系统调用包括：

- `sys_socket`、`sys_bind`、`sys_connect`
- `sys_sendto`、`sys_recvfrom`
- `sys_listen`、`sys_accept`

3. 文件描述符融合设计

将 `socket` 抽象为 `FD_SOCKET` 类型纳入文件描述符表管理，复用文件操作接口：

- 用户态调用 `read/write` 操作 `socket` 时，内核自动映射为 ONPS 的 `recv/send` 接口（逻辑见 `src/fs/file.c`）；
- 调用 `close` 关闭 `socket` 文件描述符时，内核触发 `socket` 资源的回收逻辑。

4. 错误码语义对齐

ONPS 返回的原生错误码，在内核层统一映射为 Linux 风格的 `errno`，确保用户态程序感知到的错误码语义、行为与 Linux 系统一致。

3.5 数据路径

RuOS 通过 `virtio-net` 驱动与虚拟网卡交互，ONPS 协议栈处理网络协议逻辑。具体发送与接收路径如下：

- 发送：用户态 `socket` → 系统调用层 → ONPS `socket` → IP/以太网封装 → `net_emac_send` → `virtio_net_transmit`；
- 接收：`virtio-net` 中断 → `virtio_net_rx` → `net_rx_deliver` → `ethernet_put_packet` → ONPS → 用户态 `recv/recvfrom`。

网络数据的发送与接收流程如图 7 所示：

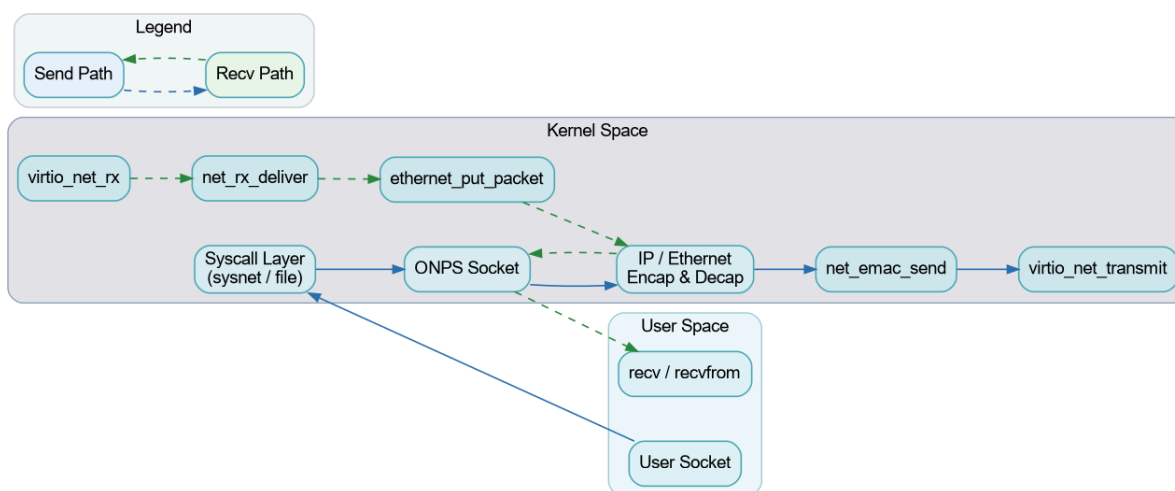


图 7: RuOS 网络数据路径图

4 程序装载

RuOS 支持 ELF 可执行文件的装载与 **动态链接**，完善了 **用户栈参数传递** 与解释器装载等细节，提升了与 Linux 用户态程序的兼容性。

4.1 ELF 文件格式

ELF (Executable and Linkable Format) 是一种通用的文件格式，用于存储可执行文件、目标代码和共享库。ELF 文件由多个部分组成，主要包括：

- **ELF 头 (ELF Header)**：包含文件的基本信息，如类型、架构、入口点地址等。
- **程序头表 (Program Header Table)**：描述了程序的各个段 (segments)，如代码段、数据段等，以及它们在内存中的加载地址和权限。
- **节头表 (Section Header Table)**：描述了文件的各个节 (sections)，如符号表、字符串表等。
- **段 (Segments)**：用于运行时加载的部分，如可执行代码段、数据段等。
- **节 (Sections)**：用于链接和调试的部分，如符号表、重定位信息等。

我们选取 2024 年初赛 SD 卡上一个 ELF 可执行文件 (“/musl/ltp/testcases/bin/waitpid01”) 作为示例：

在终端中执行：

```
1 readelf -h /musl/ltp/testcases/bin/waitpid01
```

输出结果如下：

1	Magic:	7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
2	Class:	ELF64
3	Data:	2's complement, little endian
4	Version:	1 (current)
5	OS/ABI:	UNIX - System V
6	ABI Version:	0
7	Type:	DYN (Position-Independent Executable file)
8	Machine:	RISC-V
9	Version:	0x1
10	Entry point address:	0x6684
11	Start of program headers:	64 (bytes into file)
12	Start of section headers:	798296 (bytes into file)
13	Flags:	0x5, RVC, double-float ABI
14	Size of this header:	64 (bytes)
15	Size of program headers:	56 (bytes)
16	Number of program headers:	7
17	Size of section headers:	64 (bytes)
18	Number of section headers:	32
19	Section header string table index:	31

可以看到 ELF 文件的入口点地址为 `0x6684`，表示程序开始执行的地址。程序头表从文件偏移 `64` 字节处开始，共有 `7` 个程序头，每个程序头大小为 `56` 字节。节头表从文件偏移 `798296` 字节处开始，共有 `32` 个节，每个节大小为 `64` 字节。

在所有段中，我们这里重点关注以下几个段：

- `.text` 段：包含程序的可执行代码。
- `.data` 段：包含已初始化的全局变量和静态变量
- `.bss` 段：包含未初始化的全局变量和静态变量。
- `.rodata` 段：包含只读数据，如字符串常量等。

在动态链接的 ELF 文件中，还会包含一个特殊的段：`PT_INTERP` 段。它指定动态链接器的路径，用于在程序加载时进行动态链接。

我们在终端里面执行以下命令查看 `PT_INTERP` 段的信息：

```
1 readelf -l /musl/ltp/testcases/bin/waitpid01 | grep ".interp" 
```

`-l` 选项用于显示程序头表，`grep ".interp"` 用于过滤出包含 `.interp` 段的信息。

输出结果如下：

```
1 [Requesting program interpreter: /lib/ld-musl-riscv64.so.1]
2 01      .interp
3
02      .interp .hash .gnu.hash .dynsym .dynstr .rela.dyn .rela.plt .plt .text .r
```

可以看到 `PT_INTERP` 段指定的动态链接器路径为 `/lib/ld-musl-riscv64.so.1`。这意味着在加载该 ELF 文件时，系统会使用该动态链接器来处理动态链接的需求。

至于 `.dynamic` 段和 `.rela.dyn` 段，它们也在动态链接中起着重要作用，但是相关工作由动态链接器负责处理，因此在这里我们不做过多展开。

4.2 用户栈的初始化与参数传递

用户栈的初始化与参数传递需要遵循相关 ABI 规范。在 RISC-V64 架构下，没有严格规定 `ENVP` 和 `AUXV` 的压栈方式([riscv-abi.pdf](#) 见此处)，但通常为了保持跨架构的 ABI 兼容性，我们参考了 Linux x86_64 的 ABI 规范([x86-64-abi-20210928.pdf](#))。

在 RuOS 中，我们按照以下顺序将参数压入用户栈：

- 将 `argv` 与 `envp` 的字符串内容依次拷贝到用户栈高地址处，并保持 16 字节对齐；
- 记录每个字符串的地址，组成 `argv[] / envp[]` 指针数组；
- 在指针数组之后依次放置 `auxv`（键值对形式）并以 `AT_NULL` 结束；
- 最后把 `argc` 放在栈顶，保证栈布局为 `argc | argv[] | envp[] | auxv[]`。

从高地址到低地址的实际栈布局可理解为：

- `argv` 字符串区、`envp` 字符串区（逐个写入，并对齐到 16 字节）；
- `argv[]` 指针数组（以 `NULL` 结尾）；
- `envp[]` 指针数组（以 `NULL` 结尾）；
- `auxv` 键值对数组（`AT_PHDR/...`，以 `AT_NULL` 结束）；

- `argc`。

其中 `auxv` 用于向用户态运行时传递 ELF 关键元信息：`AT_PHDR` / `AT_PHENT` / `AT_PHNUM` 指向程序头表，`AT_PAGESZ` 表示页大小，`AT_ENTRY` 为入口地址；若存在动态链接器，还会额外提供 `AT_BASE`（解释器加载基址）。栈顶对齐保证 `sp` 满足 RISC-V ABI 的 16 字节对齐要求。最后在 `exec` 中设置 `a0=argc`、`a1=argv`、`a2=envp`，使用户态入口可以按约定读取参数。

RuOS 用户栈布局如 图 8 所示：

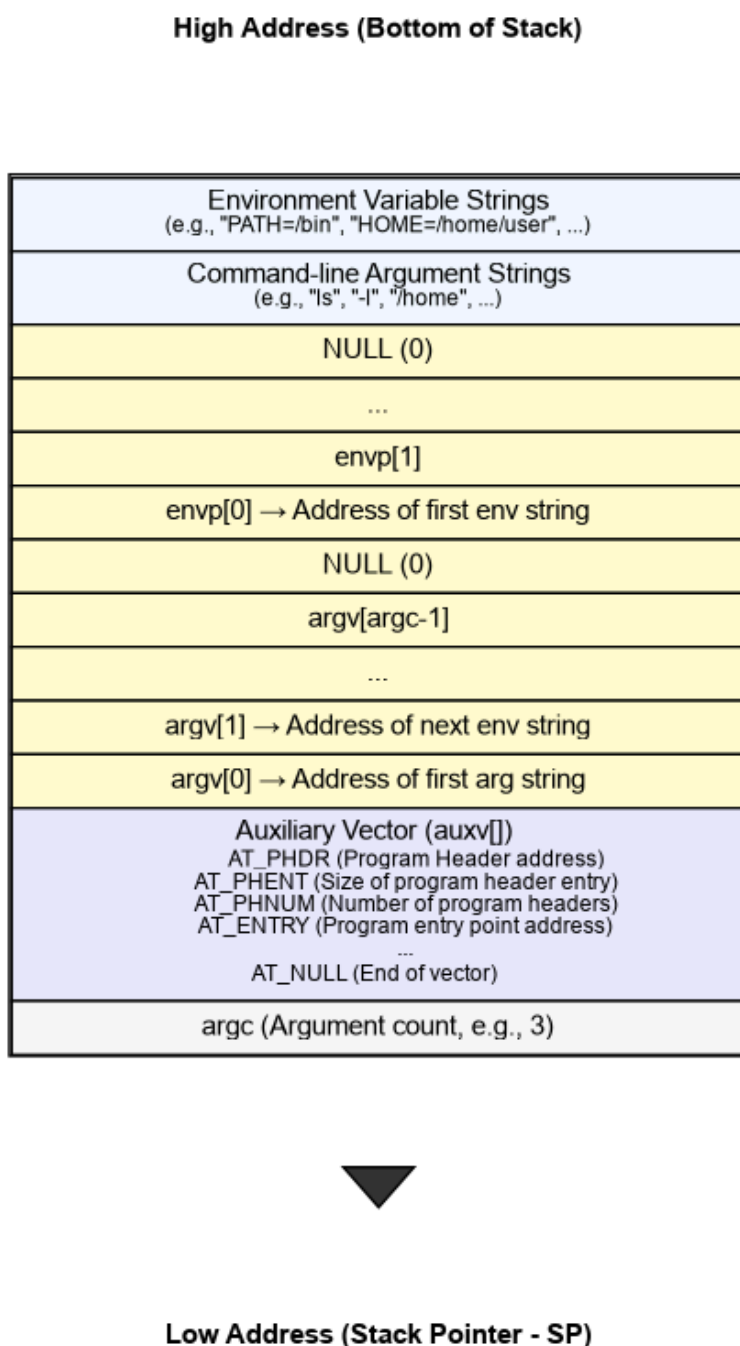


图 8: RuOS 用户栈布局图

5 系统调用

RuOS 的系统调用沿用 RISC-V/Linux 的基本约定：用户态通过 `ecall` 进入内核，系统调用号放在 `a7`，参数依次放在 `a0..a5`，返回值写回 `a0`。我们在 xv6 的基础上补齐了更接近 Linux 语义的一套系统调用，并在 `trap` 入口、参数解码、错误码返回与兼容层方面做了系统化整理，保证用户态程序（如 `busybox`）的正确执行。

与 xv6 直接硬编码 `usys.S` 不同，RuOS 在用户态封装使用了 `src/syscall/syscall.h` 中的变参宏 `syscall(...)`：通过 `__SYSCALL_NARGS` 统计参数个数，再用 `__SYSCALL_CONCAT` 选择对应的 `__syscall0..6` 内联实现。每个 `__syscallN` 把参数装入 `a0..a5`、系统调用号装入 `a7`，再执行内联 `ecall` 并返回 `a0`。这种宏分发的写法让上层接口像普通 C 函数一样调用，同时自动处理参数个数与类型提升（`__scc` 统一转为 `long`），避免为不同参数数目编写重复包装代码。

示例代码如下：

```
1  #define __asm_syscall(...) \
2      __asm__ __volatile__("ecall\n\t" \
3                          : "=r"(a0) \
4                          : __VA_ARGS__ \
5                          : "memory"); \
6      return a0;
7
8  static inline long __syscall0(long n)
9  {
10     register long a7 __asm__("a7") = n;
11     register long a0 __asm__("a0");
12     __asm_syscall("r"(a7))
13 }
14
15 static inline long __syscall1(long n, long a)
16 {
17     register long a7 __asm__("a7") = n;
18     register long a0 __asm__("a0") = a;
19     __asm_syscall("r"(a7), "0"(a0))
20 }
21
22 static inline long __syscall2(long n, long a, long b)
23 {
24     register long a7 __asm__("a7") = n;
25     register long a0 __asm__("a0") = a;
26     register long a1 __asm__("a1") = b;
27     __asm_syscall("r"(a7), "0"(a0), "r"(a1))
28 }
29
30 #define __syscall1(n, a) __syscall1(n, __scc(a))
```

```

31 #define __syscall2(n, a, b) __syscall2(n, __scc(a), __scc(b))
32 #define __syscall3(n, a, b, c) __syscall3(n, __scc(a), __scc(b),
    __scc(c))
33
34 #define __SYSCALL_NARGS_X(a, b, c, d, e, f, g, h, n, ...) n
35 #define __SYSCALL_NARGS(...) __SYSCALL_NARGS_X(__VA_ARGS__, 7, 6, 5, 4,
    3, 2, 1, 0, )
36 #define __SYSCALL_CONCAT_X(a, b) a##b
37 #define __SYSCALL_CONCAT(a, b) __SYSCALL_CONCAT_X(a, b)
38 #define __SYSCALL_DISP(b, ...) \
39     __SYSCALL_CONCAT(b, __SYSCALL_NARGS(__VA_ARGS__)) \
40     (__VA_ARGS__)
41
42 #define __syscall(...) __SYSCALL_DISP(__syscall, __VA_ARGS__)
43 #define syscall(...) __syscall(__VA_ARGS__)
44
45 #endif // __SYSCALL_LL_E

```

5.1 调用路径概览

系统调用是用户态陷入内核态的最常见路径，其核心链路如下（对应 `src/trap/` 与 `src/syscall/`）：

- 用户态执行 `ecall`，硬件跳转到 `TRAMPOLINE` 映射上的 `uservec`；
- `uservec` 保存寄存器到 `trapframe`，切换到内核页表与内核栈，再跳转到 `usertrap()`；
- `usertrap()` 识别 `scause==ECODE_SYSCALL`，手动 `epc+=4` 跳过 `ecall`，开中断后调用 `syscall_handler()`；
- `syscall_handler()` 根据 `a7` 在 `syscalls[]` 表找到 `sys_*` 处理函数，执行后把返回值写回 `a0`；
- `usertrapret()` 负责恢复 `stvec`、`sstatus`、`sepc` 并跳转 `userret`，最终 `sret` 返回用户态。

系统调用与外设中断共享同一套陷阱框架：在 `usertrap()` 中，`devintr()` 先处理外设/时钟中断，若是时钟中断则在返回前 `yield()` 让出 CPU；这保证了系统调用不会阻塞调度器，并且中断处理的时序可控。

系统调用全流程如图 9 所示（包含可选 `trace/` 统计与异常处理分支）：

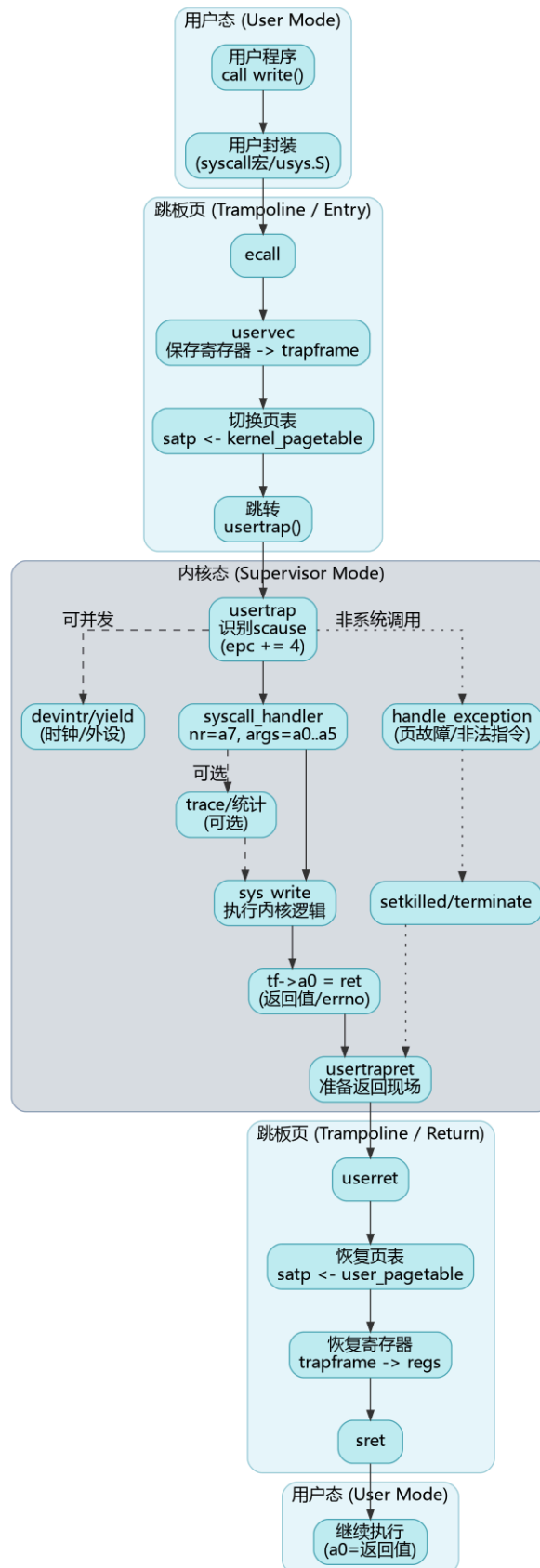


图 9: 系统调用全流程

5.2 TRAMPOLINE 与 trapframe 机制

TRAMPOLINE 与 `trapframe` 的设计解决了“切换时地址必须一直有效、且用户不可伪造上下文”的关键问题：陷入发生时 CPU 仍在用户页表下执行，必须有一段在用户页表与内核页表中都映射到同一虚拟地址的入口代码；同时寄存器保存区必须对内核可

写、对用户不可写，否则用户可篡改返回现场或内核信息。将 TRAMPOLINE 固定映射到最高虚拟地址页、trapframe 固定映射到其下一页且 `PTE_U=0`，既保证陷入/返回路径的地址稳定，又避免在每次陷入时复制寄存器结构体，兼顾安全性与性能。

为了在用户态与内核态之间快速、安全地切换，RuOS 采用与 xv6 类似的 TRAMPOLINE 设计：

- TRAMPOLINE 映射在最高虚拟地址页 (`MAXVA - PGSIZE`)，所有进程共享这页代码，用于 `uservec/userret`；
- TRAPFRAME 映射在 TRAMPOLINE 下方一页 (`MAXVA - 2*PGSIZE`)，每个进程独占一页，用户不可访问 (`PTE_U=0`)。

`trapframe` 中保存了用户态寄存器与返回上下文，同时还包括内核态需要的“跳板信息”（内核页表、内核栈顶、`usertrap` 入口、`hartid`）。`uservec` 先把用户寄存器写入 `trapframe`，再切换 `satp` 到内核页表；`userret` 在返回时反向恢复寄存器并 `sret` 回到用户态。这种布局的好处是：内核可以在不信任用户态内存的前提下完成寄存器搬运，同时避免每次陷入都复制结构体。

5.3 参数传递与用户指针解码

系统调用参数通过寄存器传递：`a0..a5` 为 6 个参数槽，`a7` 为系统调用号。`src/syscall/syscall.c` 中提供了一套统一的参数解码函数：

- `argraw(n)` 直接读取 `trapframe` 中的寄存器；
- `argint/argaddr/argstr` 在其基础上做类型转换与拷贝；
- `fetchaddr/fetchstr` 负责从用户页表中安全地 `copyin/copyinstr`。

这一层“参数解码 + 安全拷贝”的抽象很关键：它把用户指针与内核指针严格隔离，所有用户内存访问都必须经过 `copyin/copyout`。因此即使用户传入非法地址，内核也只会返回 `-EFAULT`，而不会发生越界访问。

5.4 系统调用分发与返回

系统调用分发由 `syscall_handler()` 完成，其逻辑非常直接：

- 从 `trapframe->a7` 取系统调用号；
- 在 `syscalls[]` 表中找到对应 `sys_*` 函数并执行；
- 返回值写回 `trapframe->a0`，作为用户态的系统调用返回值。

值得注意的是：`usertrap()` 会手动把 `epc` 加 4，这是为了跳过 `ecall` 指令本身，避免回到用户态后再次触发陷入；这也是 RISC-V 软件处理系统调用的通用做法。

在实现上，RuOS 保留了统一的跟踪开关 `syscall_trace_all`（默认关闭），便于在 GDB 中快速启用系统调用日志；此外还保留了错误打印与返回码映射路径，用于调试用户态程序兼容性。

5.5 系统调用集合

RuOS 实现了接近 Linux 语义的系统调用集合，涵盖文件系统、进程管理、内存管理、时间管理与网络通信等核心功能。主要系统调用包括：

- 进程/线程： `fork`、`clone`、`execve`、`exit/exit_group`、`wait/wait4`、`getpid/getppid`、`set_tid_address`；
- 内存管理： `brk/sbrk`、`mmap/munmap`、`mprotect`、`msync`；
- 文件与目录： `open/openat`、`close`、`read/write/writev`、`lseek`、`fstat/fstatat`、`mkdir/mkdirat`、`chdir/getcwd`、`unlinkat`、`fcntl`；
- 时间与定时： `sleep`、`nanosleep`、`gettimeofday`、`clock_gettime`、`times`、`setitimer`；
- 信号与调度： `rt_sigaction`、`rt_sigprocmask`、`rt_sigreturn`、`kill/tkill/tgkill`、`sched_yield`、`sched_getaffinity`；
- 管道与重定向： `pipe2`、`dup/dup2/dup3`；
- 系统/挂载： `uname`、`mount`、`umount2`、`shutdown`、`prlimit64`；
- 网络： `socket`、`bind`、`connect`、`listen`、`accept`、`sendto`、`recvfrom`、`sendfile`、`ppoll`、`ioctl`。

5.6 错误码与语义对齐

为了尽量与 Linux 用户态兼容，RuOS 的系统调用返回值遵循“成功返回非负，失败返回负 `errno`”的约定。内核中每个 `sys_*` 会根据底层模块的错误返回值进行映射：

- 文件系统与进程管理：直接返回 `-EINVAL/-ENOENT/-EFAULT/-ENOMEM` 等；
- 网络子系统：ONPS 的错误码会通过 `onps_err_to_errno()` 映射为 Linux `errno`；
- 不支持或未实现的调用：返回 `-ENOTSUP` 或 `-EINVAL`。

部分 RuOS 已经支持的错误码映射如下：

1	<code>#define</code>	<code>EPERM</code>	1	/* Operation not permitted */	
2	<code>#define</code>	<code>ENOENT</code>	2	/* No such file or directory */	
3	<code>#define</code>	<code>EINTR</code>	4	/* Interrupted system call */	
4	<code>#define</code>	<code>EIO</code>	5	/* I/O error */	
5	<code>#define</code>	<code>ENODEV</code>	19	/* No such device */	
6	<code>#define</code>	<code>ENOTDIR</code>	20	/* Not a directory */	
7	<code>#define</code>	<code>EISDIR</code>	21	/* Is a directory */	
8	<code>#define</code>	<code>EINVAL</code>	22	/* Invalid argument */	
9	<code>#define</code>	<code>EMFILE</code>	24	/* Too many open files */	
10	<code>#define</code>	<code>ENFILE</code>	23	/* File table overflow */	
11	<code>#define</code>	<code>ENOTTY</code>	25	/* Inappropriate ioctl for device */	
12	<code>#define</code>	<code>EACCES</code>	13	/* Permission denied */	
13	<code>#define</code>	<code>EFAULT</code>	14	/* Bad address */	
14	<code>#define</code>	<code>ENOMEM</code>	12	/* Out of memory */	
15	<code>#define</code>	<code>EAGAIN</code>	11	/* Resource temporarily unavailable */	
16	<code>#define</code>	<code>EWOULDBLOCK</code>	11	/* Operation would block (same as EAGAIN) */	
17	<code>#define</code>	<code>EBADF</code>	9	/* Bad file descriptor */	
18	<code>#define</code>	<code>ECHILD</code>	10	/* No child processes */	
19	<code>#define</code>	<code>ESRCH</code>	3	/* No such process */	
20	<code>#define</code>	<code>ENOEXEC</code>	8	/* Exec format error */	
21	<code>#define</code>	<code>E2BIG</code>	7	/* Argument list too long */	

```
22 #define ENOSYS 38 /* Function not implemented */
23 #define ENOTSUP 95 /* Operation not supported */
24 #define ESPIPE 29 /* Illegal seek */
```

这一层错误码语义是 `busybox` 等程序正常运行的关键：用户态不需要理解内核内部细节，只需按 POSIX/LINUX 的方式判断返回值即可。

5.7 调试与扩展点

系统调用相关的扩展与调试主要集中在以下位置：

- `syscall_handler()`：增加特定进程 `tracing`、参数和返回值打印等功能；
- `arg*` / `copyin`：可增加用户指针合法性检查或访问统计；
- `handle_exception()`：可扩展页故障恢复策略（如懒分配、COW）；
- `syscalls[]`：新增系统调用时只需注册入口并实现 `sys_*` 即可。

通过这些扩展点，RuOS 能在保持内核结构清晰的前提下逐步完善 Linux 语义与用户态兼容性，这也是我们在比赛过程中快速迭代系统调用的关键工程方法。

6 设备驱动

RuOS 运行在 QEMU `virt` 平台上，主要外设都以 memory-mapped I/O (MMIO) 的形式暴露：CPU 通过读写固定物理地址处的寄存器与设备交互；当设备需要“打断”内核（例如收到字符、完成 DMA、队列有回收项）时，再通过外部中断通知内核。与真实硬件相比，`virt` 平台的好处是设备模型稳定、可复现且便于调试，但也要求驱动严格遵守 MMIO 的时序与内存序，否则很容易出现“偶发丢中断/丢包/卡死”等难以定位的问题。

本章围绕 `src/devs/uart.c`、`src/devs/plic.c` 与 `src/virtIO/`，从 MMIO、PLIC、中断分发到 VirtIO virtqueue/ring（包括 `tx_ring` 的发送完成回收），系统性说明 RuOS 的设备驱动基础知识与实现细节。

6.1 MMIO：设备寄存器与内存序

在 RISC-V 上，MMIO 通常表现为一段“不可缓存、具有副作用”的地址区间：读寄存器可能清状态、写寄存器可能触发发送/通知。RuOS 在实现上采用两条简单但关键的规则：

- **volatile 访问**：通过 `*(volatile uint8*)/*(volatile uint32*)` 访问寄存器，避免编译器把读写优化掉或合并重排（例如 `src/devs/uart.c` 的 `UART_WRITE_REG/UART_READ_REG`，以及 `src/virtIO/virtio_*.c` 的 `R(n, reg)` 宏）。
- **显式内存屏障**：在把描述符/环形队列写入内存并“通知设备”之前，必须先保证这些内存写对设备可见；反之，在处理中断时读取 `used ring`，也要避免乱序导致看到旧数据。RuOS 广泛使用 `__sync_synchronize()` 作为保守的全栅栏，确保“先写队列，再 kick；先读状态，再回收”的顺序成立。

QEMU `virt` 的关键 MMIO 地址在 `src/memlayout.h` 中定义。这些物理地址在内核页表中保持可访问，因此驱动可以直接以物理地址形式进行寄存器读写）。

6.2 设备与中断拓扑

外设的数据通路与中断通路通常是“分离”的：数据通路走 MMIO/DMA，中断通路走 PLIC。RuOS 的总体拓扑如图 10 所示：

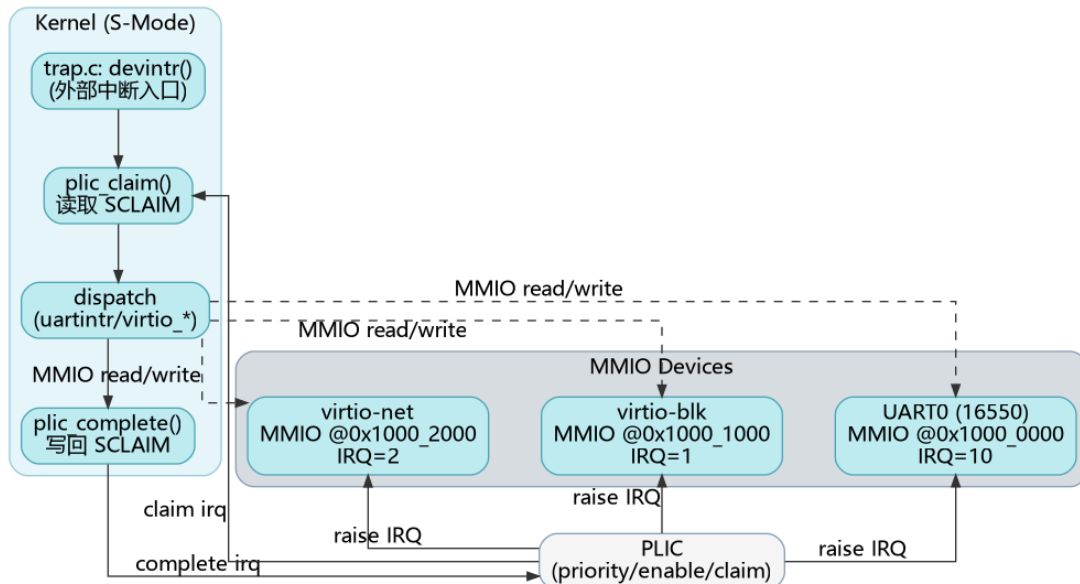


图 10: RuOS 设备与中断拓扑 (MMIO 数据通路 + PLIC 中断通路)

在启动阶段 (`src/boot/main.c`), `hart0` 完成全局初始化: `consoleinit()` 初始化 UART; `plicinit()` 设置外设 IRQ 的优先级; 随后每个 `hart` 都会调用 `plicinithart()` 配置自己的 S-mode 使能与阈值, 并打开 `SIE_SEIE` (Supervisor External Interrupt Enable), 从而允许设备中断进入 `devintr()`。

6.3 UART: 控制台与早期调试

UART 是最朴素也最重要的外设之一: 它提供早期打印、交互式 shell 输入输出, 是定位启动问题和内核崩溃的“生命线”。RuOS 使用 QEMU virt 默认的 16550 兼容 UART (`UART0@0x10000000`), 驱动实现位于 `src/devs/uart.c`, 并由 `src/devs/console.c` 提供行缓冲与用户态 `read/write` 的对接。

UART 驱动的关键点包括:

- 初始化: `uartinit()` 先关闭中断, 再配置波特率锁存 (`LCR_BAUD_LATCH`) 与分频因子, 设置 8-bit 数据位 (`LCR_EIGHT_BITS`), 最后打开 FIFO 并把 RX 触发阈值设为 14 字节 (减少单字符中断), 再开启 RX/TX 中断 (`IER_RX_ENABLE/IER_TX_ENABLE`)。
- 输出路径: `uart_write_byte_nolock()` 通过轮询 `LSR_TX_IDLE` 等待发送端空闲, 再写 `THR` 发送字节; 上层的 `uart_write/uartputc_sync` 用 `uart_tx_lock` 串行化输出, 保证并发 `printf` 不会互相穿插。panicked 时直接自旋, 避免继续输出导致更多状态破坏。
- 输入路径: `uartintr()` 在外部中断到来后持续读取 `RHR` (直到 `LSR` 表明无数据), 把字符交给 `consoleintr()`。控制台层维护一个小型环形缓冲 (`INPUT_BUF_SIZE=128`), 支持退格、回车换行、Ctrl-D EOF 等语义, 并通过 `sleep/wakeup` 让阻塞的 `consoleread()` 在“整行到达”后被唤醒。

这种设计把“慢速字节流设备”和“面向行的用户交互”分层: UART 专注于寄存器与中断, Console 专注于缓冲与语义, 这也是 RuOS 后续接入更多 TTY/伪终端能力的良好起点。

6.4 PLIC: 外部中断控制器

PLIC (Platform Level Interrupt Controller) 负责把多个外设的 IRQ 汇聚到各个 hart, 并提供优先级、屏蔽与 claim/complete 的握手机制。对驱动开发而言, 理解 PLIC 的三个接口就足够:

- **priority**: 优先级为 0 表示禁用; 非 0 表示可投递。RuOS 在 `plicinit()` 中为 `UART0_IRQ/VIRTIO0_IRQ/VIRTIO1_IRQ` 写入优先级 1 (`src/devs/plic.c`)。
- **enable/threshold (按 hart)**: 每个 hart 有独立的 `PLIC_SENABLE(hart)` 位图与 `PLIC_SPRIORITY(hart)` 阈值。RuOS 在 `plicinithart()` 中把 UART、virtio-blk、virtio-net 的 enable 位都置 1, 并把阈值设为 0 (即允许投递所有非 0 优先级的中断)。
- **claim/complete (按 hart)**: 当发生 supervisor external interrupt 时, `devintr()` 调用 `plic_claim()` 读取 `PLIC_SCLAIM(hart)` 获得一个 IRQ 号; 处理完成后必须调用 `plic_complete(irq)` 把相同的 IRQ 写回 claim 寄存器, PLIC 才会允许该设备再次触发中断。

RuOS 的中断分发逻辑在 `src/trap/trap.c:devintr()`: 根据 claim 到的 IRQ 号调用 `uartintr()`、`virtio_disk_intr()` 或 `virtio_net_intr()`, 最后统一 complete。这种 “claim->dispatch->complete” 的模板使得新增设备非常直接: 只需在 PLIC 侧启用对应 IRQ, 并在 `devintr()` 增加分支即可。

PLIC 的工作机制如图 11 所示:

Platform-Level Interrupt Controller

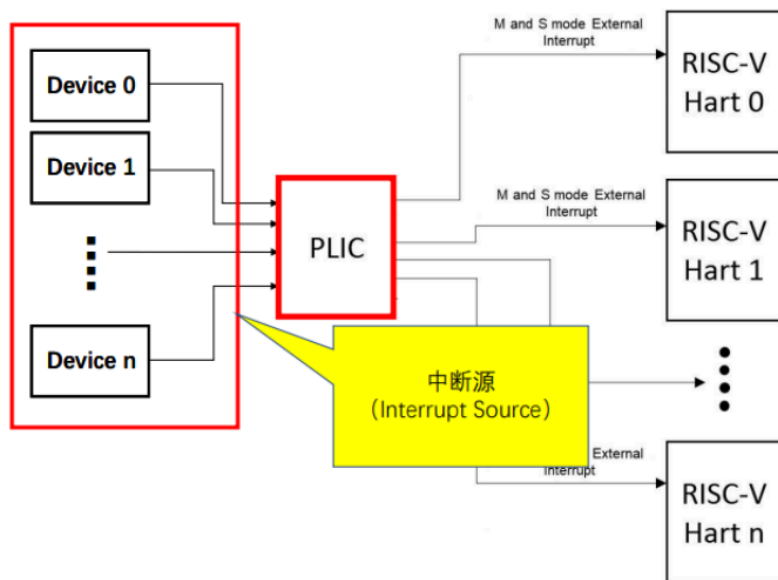


图 11: PLIC 中断工作机制

6.5 VirtIO: 半虚拟化设备与 mmio legacy 接口

VirtIO 是 QEMU virt 平台上常用的半虚拟化设备规范, 核心思想是: 驱动与设备共享一段内存队列 (virtqueue), 通过少量 MMIO 寄存器完成特性协商、队列注册与通知。RuOS 选择实现 QEMU 的 legacy virtio-mmio 接口 (`src/virtIO/virtio.h` 给出了寄存器偏移与状态位), 其初始化流程可概括为:

- 读取 `MAGIC/VENDOR/DEVICE_ID` 做设备存在性校验;
- 依次设置 `STATUS`: `ACKNOWLEDGE` -> `DRIVER` -> `FEATURES_OK` -> `DRIVER_OK`;
- 读取 `DEVICE_FEATURES` 并清掉不支持/不需要的特性 (如 `INDIRECT_DESC` 等), 再写回 `DRIVER_FEATURES`;
- 配置页大小 `GUEST_PAGE_SIZE=PGSIZE`;
- 对每个队列: 写 `QUEUE_SEL` 选择队列, 检查 `QUEUE_NUM_MAX`, 设置 `QUEUE_NUM`, 为队列分配一段连续、页对齐的内存, 并把其 `PFN` 写入 `QUEUE_PFN`。

RuOS 的 `virtio-blk` 与 `virtio-net` 都采用 “2 页队列布局”:

- 第一页放 `desc[]` 与 `avail[]`
- 第二页放 `used[]`

具体代码见 `src/virtIO/virtio_disk.c` 与 `src/virtIO/virtio_net.c` 的 `pages[2*PGSIZE] / rx_pages/tx_pages`。

6.6 virtqueue 与 ring: 从入队到回收

`virtqueue` 的数据结构由三部分组成: 描述符表 `desc[]`、驱动侧可用环 `avail`、设备侧完成环 `used`。其典型交互过程如图 12 所示:

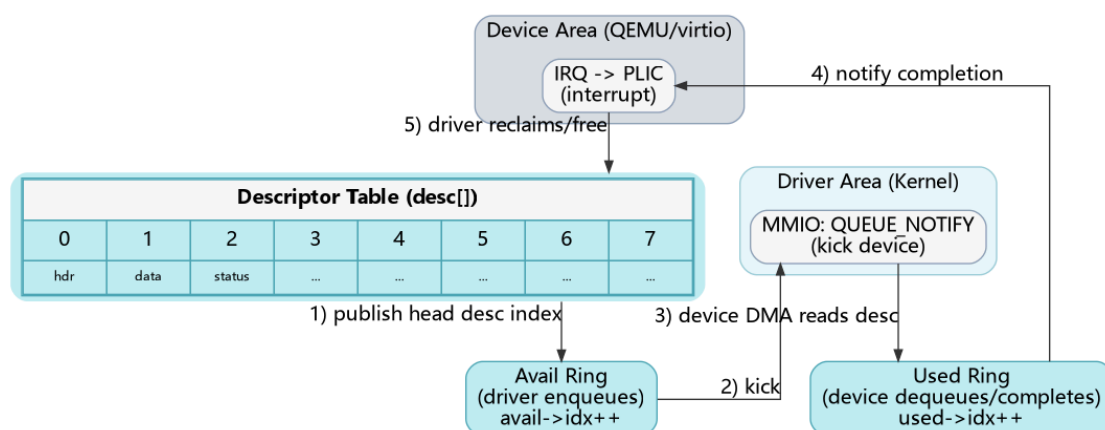


图 12: VirtIO virtqueue/ring 基本交互

其中最容易踩坑的点是内存可见性与索引推进:

- 驱动需要先把 `desc[]` (DMA 地址、长度、flags、next) 写好, 再把链表头索引写入 `avail[2 + (avail->idx % NUM)]`, 最后在内存屏障之后递增 `avail->idx` 并写 `QUEUE_NOTIFY`;
- 设备处理完成后会更新 `used->idx` 并写入 `used->elems[]` (包含已完成链表头 id 与长度), 随后触发 IRQ;
- 驱动在中断上下文中从 `used_idx` 扫描到 `used->idx`, 依次回收 descriptor 链并唤醒等待者。

所谓 `tx_ring` (发送环) 本质上就是 “用于发送方向的 `virtqueue`”: 以 `virtio-net` 为例, 队列 1 作为 TX queue, 驱动把 “报文头 + 报文数据” 的 descriptor 链入队; 设备把完成项写入 `used ring`; 驱动在 `virtio_net_tx_complete()` 里根据 `used ring` 回收链表并释放对应缓冲, 从而实现发送完成的资源回收与背压控制。

6.7 virtio-blk: 块设备 I/O

RuOS 的 virtio-blk 驱动采用中断完成的同步模型：发起 I/O 的线程在睡眠中等待中断唤醒。每次读写会构造 3 个 descriptor 的链表，这是 legacy virtio-blk 的标准做法：

- descriptor0: 请求头 `virtio_blk_outhdr` (type/sector 等)；由于该结构体在内核栈上，需用 `kvmppa()` 转成可 DMA 的物理地址；
- descriptor1: 数据缓冲区 `b->data` (读时 `VRING_DESC_F_WRITE` 让设备写入，写时 `flags=0` 让设备读取)；
- descriptor2: 1 字节 status (设备写入完成状态)。

驱动用 `vdisk_lock` 保护 free list、`avail/used` 指针与 `used_idx`，当 descriptor 不足时对 `free[0]` 睡眠等待；发起请求后把链表头塞入 `avail ring` 并 `notify`，然后在 `b->disk==1` 时对 `b` 睡眠。中断处理函数 `virtio_disk_intr()` 扫描 `used ring`：校验 status、清除 `b->disk`、唤醒等待该 buf 的线程，并回收整个 descriptor 链。该模型简单可靠，足以支撑文件系统与 `busybox` 的大量 I/O。

6.8 virtio-net: 收发队列与协议栈对接

virtio-net 相比块设备更强调吞吐与并发，RuOS 采用 **双队列** 设计：队列 0 为 RX，队列 1 为 TX。

- RX (预投递缓冲 + 回收再投递)：初始化时为每个 descriptor 预先绑定一块 `rx_buf[i]` (包含 `virtio_net_hdr` + payload 空间)，`flags` 置 `VRING_DESC_F_WRITE` 允许设备写入，然后把所有 descriptor 索引推入 `rx_avail` 并 `notify`。中断到来时 `virtio_net_rx()` 扫描 `rx_used`：取出完成项长度，跳过 virtio 头后把报文交给 `net_rx_deliver()`，随后把同一个 descriptor id 再次写回 `rx_avail` 实现缓冲复用，最后 `notify` 让设备继续收包。
- TX (`tx_ring` 入队 + 完成回收)：发送时 `virtio_net_transmit()` 分配 2 个 descriptor (header + data)，把链表头写入 `tx_avail` 并 `notify=1`。完成后 `virtio_net_tx_complete()` 扫描 `tx_used`：释放对应 `tx_info` 中记录的发送缓冲并 `free_chain()` 回收 descriptor。当前实现选择在 TX 完成时 `kfree(data)`，因此网络栈侧通常以“发送缓冲由内核分配、驱动负责回收”的约定避免额外拷贝；若后续需要支持零拷贝或用户态 socket 直传，可在此处引入引用计数与更细粒度的生命周期管理。

`virtio_net_intr()` 负责统一 ACK `INTERRUPT_STATUS` 并串行执行 RX/TX 回收逻辑；`vnet.lock` 保护队列状态与 free list，避免发送线程与中断处理并发修改 ring 造成索引错乱。

6.9 工程化细节与可扩展点

设备驱动往往是“最像硬件、最容易出现时序 bug”的部分。RuOS 在实现时做了几处工程化取舍以降低风险：

- 保守的 feature 协商：显式关闭 `EVENT_IDX/INDIRECT_DESC/ANY_LAYOUT` 等特性，避免实现复杂度和兼容性问题；

- 统一的同步原语：块设备走“sleep 等中断”的同步模型，网卡走“中断回收 + 上层队列”的异步模型，但二者都用自旋锁保护 ring 与 free list，保证最小正确性；
- 清晰的扩展点：新增 VirtIO 设备通常只需复用 `virtio.h` 的寄存器框架与 `virtqueue` 初始化模板；增加统计/trace 可在 `devintr()`、`virtio*_intr()`、以及每次 `notify` 前后记录时间戳与队列深度。

通过 UART+PLIC+VirtIO 的组合，RuOS 在 QEMU virt 平台上形成了一条稳定的“输入/输出/网络”最小闭环，为文件系统、网络协议栈与用户态 `busybox` 提供了必要的底层支撑；同时代码结构也保持了与 xv6 类似的清晰分层，便于比赛期间快速迭代与定位问题。